Project Manifest and Design Rationale

Project purpose

NovaHearth Warranty Case Agent is a local demonstration application for importing warranty claims, gathering warranty evidence, applying deterministic policy, recording simulated resolutions, and routing uncertain cases to human review. Refunds, credits, replacements, and reversals are ledger simulations; they do not call a payment processor or fulfillment service.

Agent and orchestration logic

Component	Location	Responsibility
Claim intake agent	`warranty_agent/agents.py - ClaimIntakeAgent`	Calls the selected chat model with a structured-output schema (`ClaimUnderstanding`) to extract remedy, customer-mentioned product, issue excerpt/category, repeat-contact signal, and ambiguity. It does not decide coverage or approve actions. Deterministic checks ground excerpts and explicit remedy wording in the submitted claim.
Warranty assessment agent	`warranty_agent/agents.py - WarrantyAssessmentAgent`	Gives the model bounded control of read-only evidence tools. It validates tool names/order, reconstructs arguments from trusted claim, CSV, and policy state, and fills only missing required evidence calls through a logged deterministic fallback. It also checks claimant-to-warranty-holder identity and assembles evidence from the claim and exact warranty row.
Resolution policy agent	`warranty_agent/agents.py - ResolutionPolicyAgent`	Applies deterministic eligibility rules. It routes missing/mismatched records, unclear or unsupported issues, repeat replacements, invalid values, configured approval requirements, and values at or above the policy limit to human review. It alone decides whether the workflow may auto-resolve.
Workflow graph	`warranty_agent/workflow.py - WarrantyWorkflowFactory`	Builds the LangGraph state machine: intake -> assessment -> policy -> either action or a durable human interrupt -> action or denial. It records step events and retries eligible transient failures; provider rate-limit failures are not retried.
Shared state and schemas	`warranty_agent/domain.py`	Defines the structured intake schema, graph state shape, and validated warranty policy model.
Provider adapters	`warranty_agent/providers.py`	Creates LangChain chat models for a configurable OpenAI-compatible in-house endpoint, Groq, OpenAI, or local Ollama. Provider/model defaults come from environment settings.

Tool definitions

Read-only evidence tools

These are declared with LangChain's `@tool` decorator in `warranty_agent/agents.py` and are called inside `WarrantyAssessmentAgent`:

- `lookup_warranty_record(product_ref)` - retrieves the warranty row by the claim's exact product reference. The claim's stored reference is used; a model-selected product cannot redirect the lookup.
- `lookup_replacement_history(customer_name)` - combines supplied CSV replacement history with replacement actions already recorded by the application. This is required for replacement intent and skipped for refund, credit, and unclear intent.
- `check_warranty_support(record_category, alleged_category, allowed_categories, issue_text)` - checks that the record category is allowed, the issue is specific, categories/evidence agree (with the explicit known-recall rule), and the customer's text does not indicate accidental damage or normal wear. The caller builds these arguments from trusted data rather than accepting the model's argument values.

The assessment model may choose when to call the bounded evidence tools, but it cannot call resolution tools. A four-turn bound, duplicate/unexpected-call checks, required-tool checks, and a deterministic missing-tool fallback keep the assessment complete before policy evaluation.

Simulated action tools

Declared in `warranty_agent/action_tools.py` and invoked by the graph only after the deterministic policy gate or a human approval:

- `issue_simulated_refund(...)` - records a simulated refund action.
- `issue_simulated_credit(...)` - records a simulated credit action.
- `schedule_like_for_like_replacement(...)` - records a simulated replacement schedule for the authoritative product reference.

The tools persist through `ClaimRepository.add_action`; they do not move money or ship goods. Reviewer approvals are recorded with the action. The UI's post-resolution denial uses the reverse endpoint and appends a separate reversal record; it does not undo an external transaction.

Persistence and interfaces

- **CSV ingestion and validation:** `warranty_agent/data_sources.py` validates required files, headers, identifiers, monetary values, and flags; it indexes exact product references and customer replacement counts.
- **Application service:** `warranty_agent/service.py` imports claims, starts/resumes graph runs, maps workflow failures to retryable claim state, and persists final graph state.
- **SQLite repository:** `warranty_agent/repositories.py` owns schema creation/migration and persistence for `claims`, `actions`, `human_reviews`, `events`, and `llm_usage`. It provides atomic claim acquisition, restart recovery, action idempotency, append-only audit events, human overrides, and simulated reversals. LangGraph checkpoints share the configured SQLite database through its SQLite checkpointer.
- **Token usage:** `warranty_agent/token_usage.py` normalizes provider-reported token metadata and records it; it does not estimate missing usage.
- **HTTP API:** `api.py` provides FastAPI routes for configuration/health, claim import/list/detail/process, background batch jobs, reviews/overrides/reversals, actions, audit events, token usage, warranties, and the demo-only reset endpoint.
- **Dashboard:** `frontend/src/App.jsx`, `frontend/src/api.js`, and `frontend/src/styles.css` implement the React workspace, API client, claim queue, agent-completed and human-review views, action/audit displays, policy view, and reset controls.
- **Configuration and data:** `policy.json` contains the policy version, strict auto-resolution limit, eligible warranty categories, and approval flags. The three source datasets are in `data/`.
- **Verification suite:** `tests/test_warranty_app.py` covers policy boundaries, tool behavior/fallback, prompt injection containment, concurrency, human review, reset, provider setup, and persistence.

Design Rationale

I chose an explicit LangGraph workflow because human review must pause and resume, and each step needs an auditable event. The LLM interprets free text and selects evidence tools; deterministic rules decide coverage, value limits, and whether an action is allowed. I use exact product-reference lookup because each claim already includes a reference and selecting the wrong warranty row would be unsafe. SQLite and CSVs keep this demo easy to run and inspect, though they are not a production scaling design.

The brief leaves "reasonable value threshold" undefined and does not specify how to value a replacement. I made the limit configurable, set the demo default to **strictly below $100**, use credit value for refunds/credits and purchase price for replacements, and route values at or above the limit to a reviewer. Repeat replacement reports or recorded prior replacements also require review. I show concise evidence and decision reasons rather than private model chain-of-thought. Since actions are simulated, a reversal is an appended audit/action record; it cannot undo a real payment or shipment.

One correction from the earlier AI-assisted iteration was removing the model's self-reported confidence field. I questioned whether it belonged: the brief does not request confidence, and the score is uncalibrated, so it should not influence approval. With more time, I would build a held-out labeled evaluation set for extraction errors and false approvals, add authentication and reviewer roles, move batch work to a durable queue, and integrate authenticated, idempotent payment and fulfillment services with reconciliation and monitoring.